CSS234 WEB PROGRAMMING II

LECTURE 01 – COURSE SYLLABUS AND REQUIREMENTS

Dr. Wittawin Susutti
wittawin.sus@kmutt.ac.th

Objectives

- Apply a structured approach to identifying needs, and functionality of a website.
- Design and create a dynamic website that meet specified needs using libraries and frameworks.
- Perform unit and integration testing.
- Validate and verify the web program developed.

Lecture

Week	Topic
1	Introduction and Requirement engineering
2	RESTful API Design
3	Alternative Communications
4	ORM
5	Project progress 1
6	Authentication & Authorization
7	Event-Driven Systems
8	Lab test #1
9	Backend Security
10	Project progress 2
11	Caching Strategies
12	Distribution Systems
13	Testing & Quality Assurance
14	Lab test 2
15	Project presentation

Lab

Week	Topic
1	Introduction and Requirement engineering
2	API
3	gRPC
4	ORM
5	Project progress 1
6	OAuth2
7	Event-Driven Systems
8	Lab test #1
9	Hashing
10	Project progress 2
11	Caching
12	Microservices
13	Manual Testing
14	Lab test 2
15	Project presentation

Assessment & Textbooks

วิธีการวัดและประเมินผล	สัดส่วนของการวัดและการประเมินผลรวม
การบ้าน และงานภาคปฏิบัติ	10%
สอบย่อย	20%
Project	30%
สอบกลางภาค และปลายภาค	15% + 15% = 30%
Web technology present	10%

Textbook:

- Pedro Marquez-Soto, Backend Developer in 30 Days: Acquire Skills on Api Designing, Data Management, Application Testing, Deployment, Security and Performance Optimization, BPB Online, 2022.
- James Higginbotham, Principles Of Web Api Design: Delivering Value With Apis And Microservices, Addison-Wesley, 2021.

Project organization

- Number of members – **TWO to THREE**, any conditions for grouping?

- Fixed topic

Tasks

- Create web app based on the topic supported by user needs or requirements

- Select one web technology: learn, summarize and present with demo (if applicable) to your friends and on discord (5 mins)

Requirements

- Before you can build something, you have to know what it is that needs building.

- Requirements is an area of software engineering concerned with the elicitation, analysis, specification, and validation of software requirements.

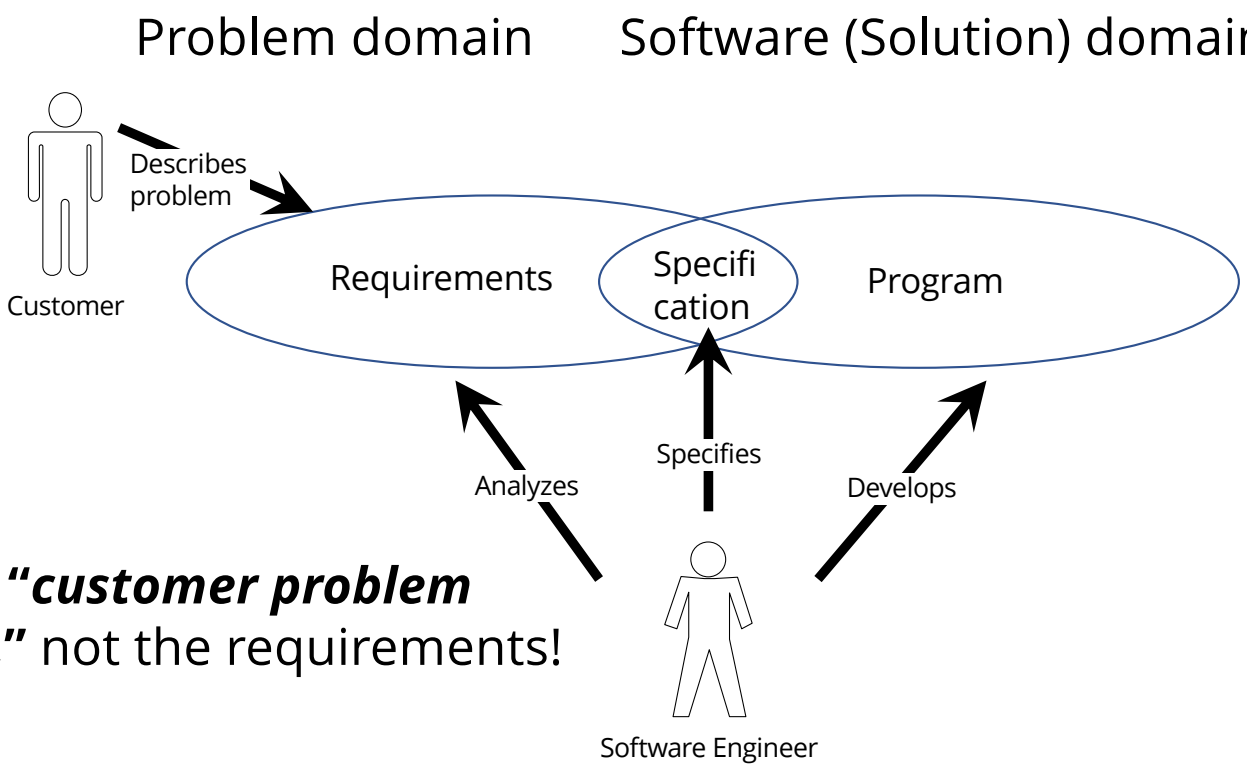
"The hardest single part of building a software system is deciding precisely what to build." [Brooks, 1987]

Definition

Requirements - Internal and external capabilities and characteristics of the system necessary to satisfy the **explicit** and **implicit** needs of the customer and/or user.

Requirements – A statement of a customer **need or objective**, or **a condition or capability** that a product must possess to satisfy such a need or objective. A property that a product must have to provide value to a stakeholder.

Requirements and Specification



We receive ***"customer problem statement,"*** not the requirements!

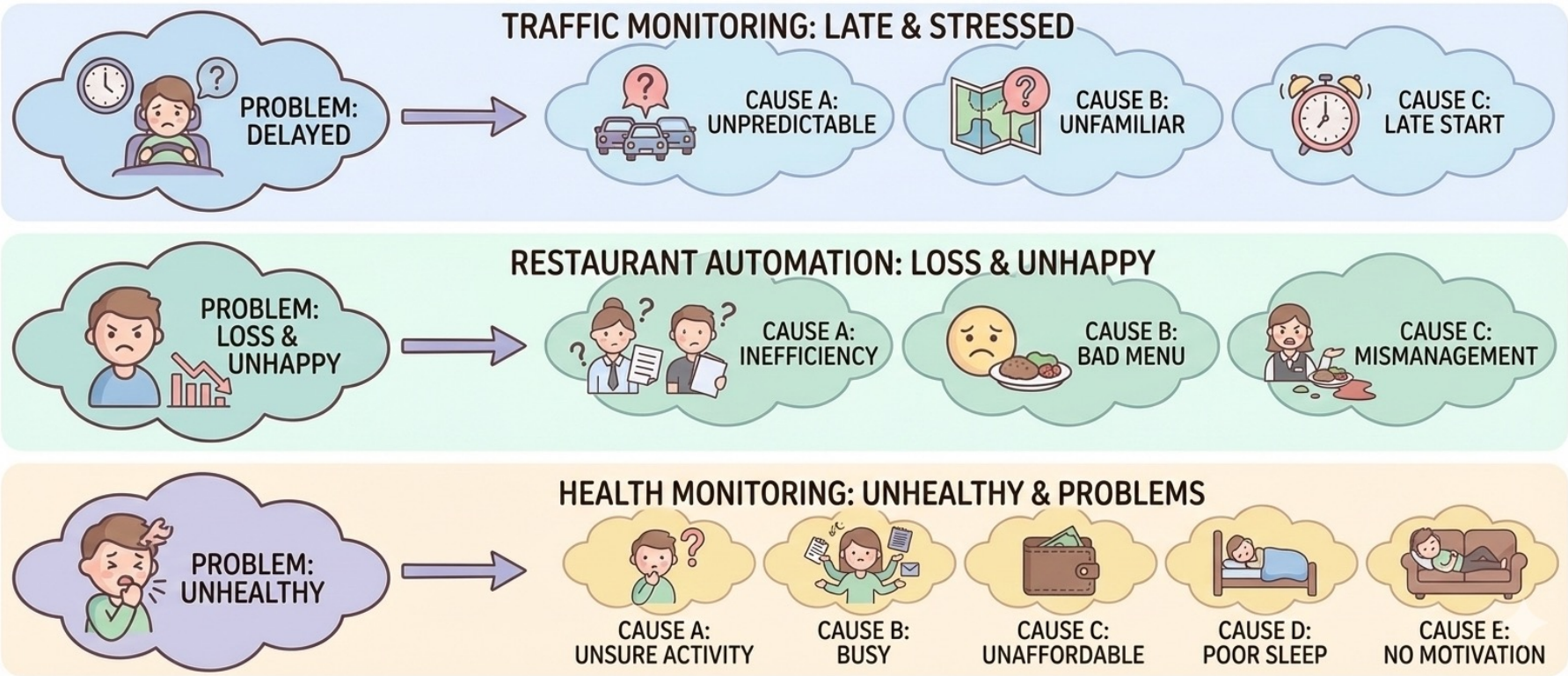
Requirements Derivation

- *Detecting* that a problem exists is different from *defining* the problem and its **causes**, and solution **constraints**.
- Depending on the cause, the solution will be different.
- Requirements are determined by:
  - Judgment about customer's business needs and causes preventing their achievement
  - Conditions on solutions imposed by real-world constrains:
    - Physical
    - Social/Cultural
    - Legal
    - Financial
    - …

Problem Analysis Examples

- **Traffic Monitoring:**
  - Problem: User is delayed to work and needs help
  - Cause A: Traffic is unpredictable (predictably high if repeated delays?!)
  - Cause B: User is unfamiliar with the route
  - Cause C: User has a habit of starting late
- **Restaurant Automation:**
  - Problem: Restaurant is operating at a loss and customers are unhappy
  - Cause A: Staff is encumbered with poor communication and clerical duties, resulting in inefficiencies
  - Cause B: The menu does not match the likeliest customer's taste
  - Cause C: Staff is misbehaving with customers or mismanaging the food supplies
- **Health Monitoring:**
  - Problem: User is leading unhealthy lifestyle and experiencing health problems
  - Cause A: User is trying to be active but unsure about sufficient level of activity
  - Cause B: User is trying to be active but too busy with other duties
  - Cause C: User cannot find affordable solutions for active lifestyle
  - Cause D: User has poor sleep
  - Cause E: User is aware of issues but not motivated to act

Problem Analysis Examples



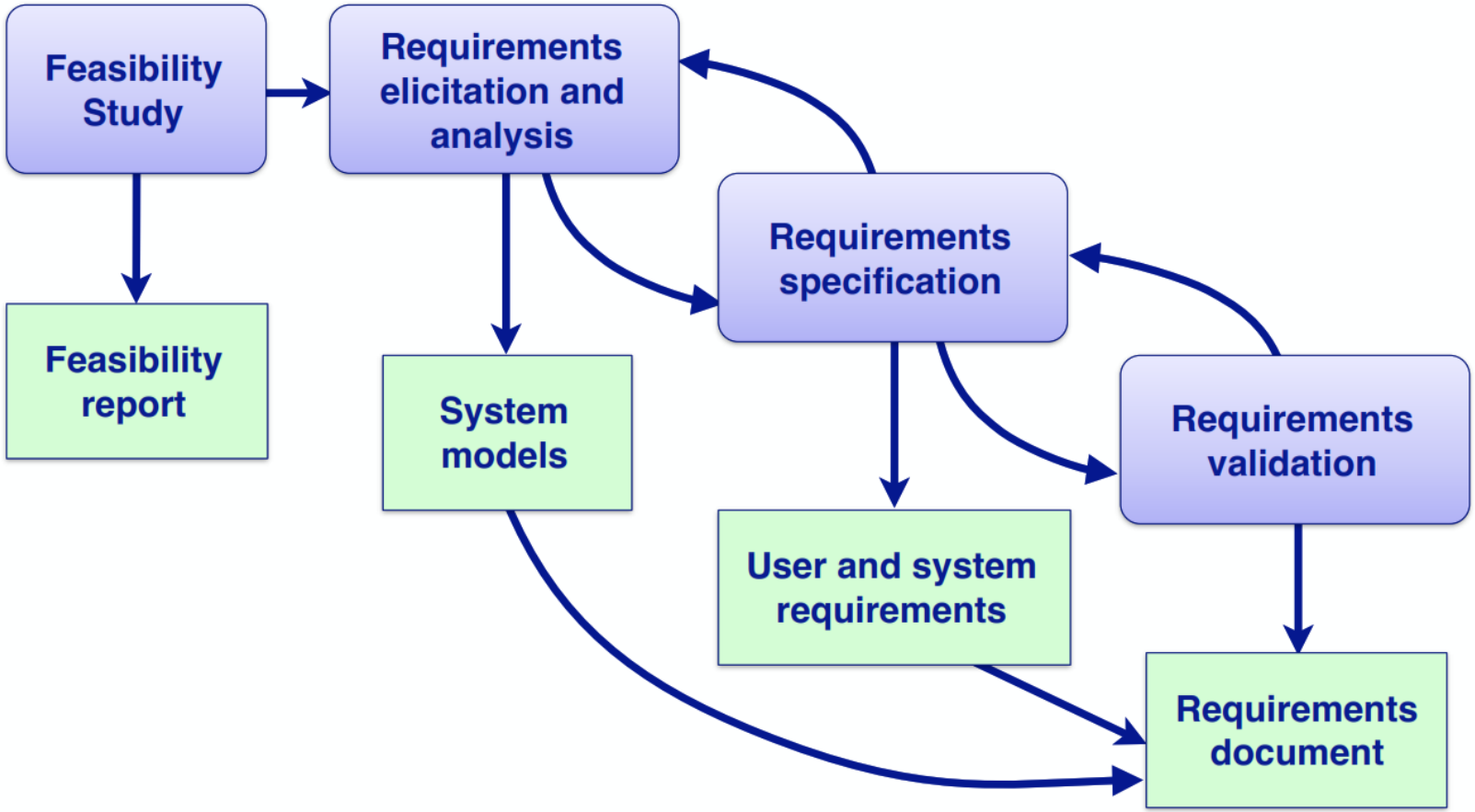
Requirements as User Stories

As a tenant, I can unlock the doors to enter my apartment.

user-role (benefactor)	capability	business-value

- Preferred tool in agile methods.
- Focus on the *user benefits*, instead on *system features* as in older, IEEE Standard 830 type “requirements”.
- User stories are *less formal* and emphasize automated acceptance tests.

The Requirements Engineering Process



Requirements Engineering Activities

Feasibility study	Determine if the *user needs* can be *satisfied* with the *available technology* and *budget*.
Requirements analysis	Find out *what system stakeholders require* from the system.
Requirements definition	*Define the requirements* in a form understandable to the customer.
Requirements specification	Define the requirements in *detail*. (Written as a contract between client and contractor.)

Requirements Analysis

- Sometimes called **requirements elicitation** or **requirements discovery**
- Technical staff work with customers to determine
  - the application domain
  - the services that the system should provide
  - the system's operational constraints
- Involves various **stakeholders**:
  - e.g., end-users, managers, engineers involved in maintenance, domain experts, etc.

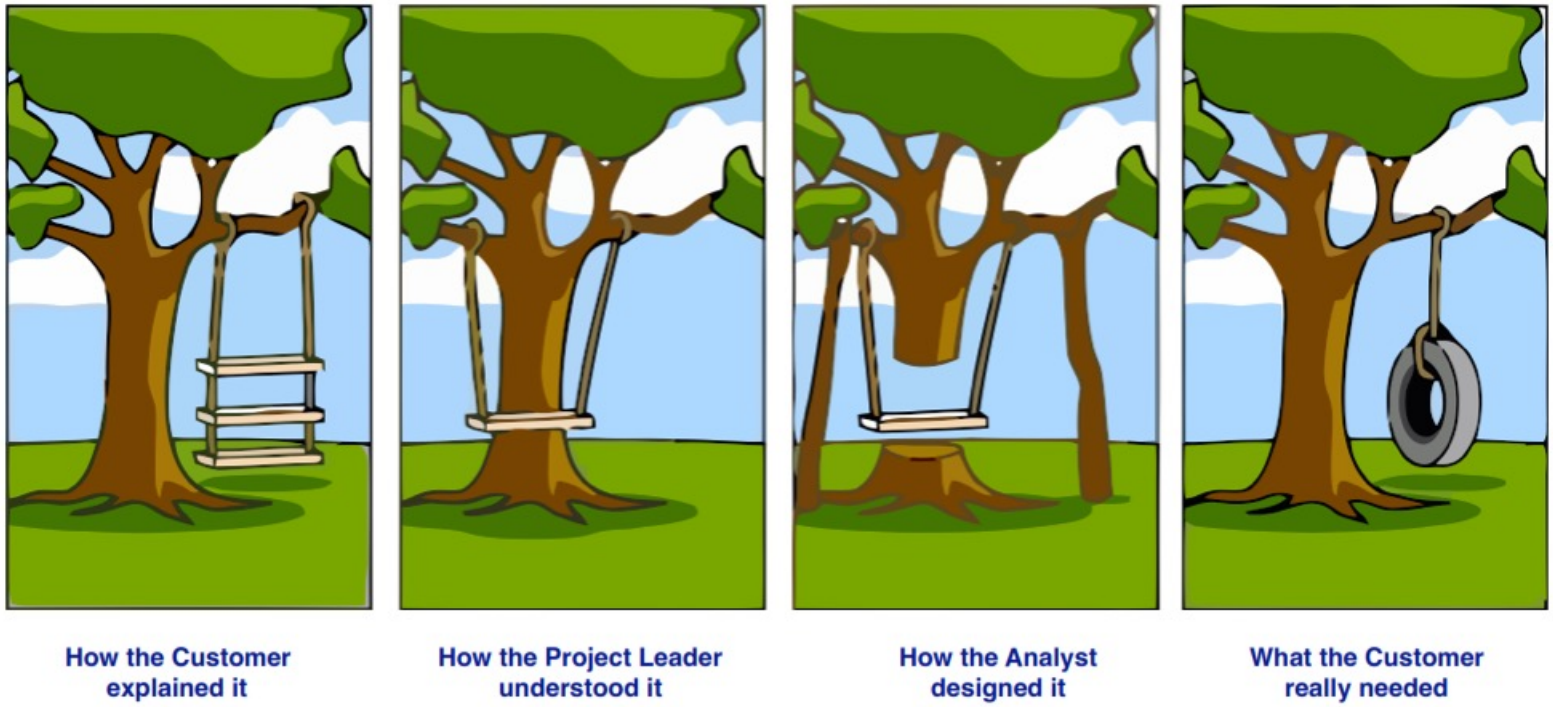
Problems of Requirements Analysis

- Various problems typically arise:
  - Stakeholders don’t know what they really want
  - Stakeholders express requirements in their own terms
  - Different stakeholders may have conflicting requirements
  - Organizational and political factors may influence the system requirements
  - The requirements change during the analysis process.
  - New stakeholders may emerge.

Problems of Requirements Analysis

- Stakeholders may not be aware of possible alternative technical solutions.
- Each person comes to the project with their own perspective and may only have a vague idea of the requirements of other stakeholders.

Problems of Requirements Analysis



- The key thing not to lose sight of is “what the customer needs”.
- Since there may be many stakeholders, it is important to figure out who is driving the project and what the business case is.

Requirements evolution

- Requirements always evolve as a better understanding of user needs is developed and as the organization's objectives change.

- It is essential to plan for change in the requirements as the system is being developed and used

Use Cases and Scenarios

- A use case is the specification of a sequence of actions, including variants, that a system (or other entity) can perform, interacting with actors of the system.
  - login to the timetable system
  - define a new committee
  - schedule a meeting

- A scenario is a particular trace of action occurrences, starting from a known initial state.
  - go to the timetable web site; enter your login; click on “forgot password” ...
  - enter “new committee”; enter committee name; select committee members from database; indicate committee chair; ...

- This approach captures how the system will be used, rather than just listing the desired “features”, which expresses only what the system should do.

Use Cases and Viewpoints

- Stakeholders represent different problem viewpoints.
  - Interview as many kinds of stakeholders as possible/necessary
  - Translate requirements into use cases or “stories” about the desired system involving a fixed set of actors (users and system objects)
  - For each use case, capture both typical and exceptional usage scenarios

- Users tend to think about systems in terms of “features”.
  - You must get them to tell you stories involving those features.
  - Use cases and scenarios can tell you if the requirements are complete and consistent!

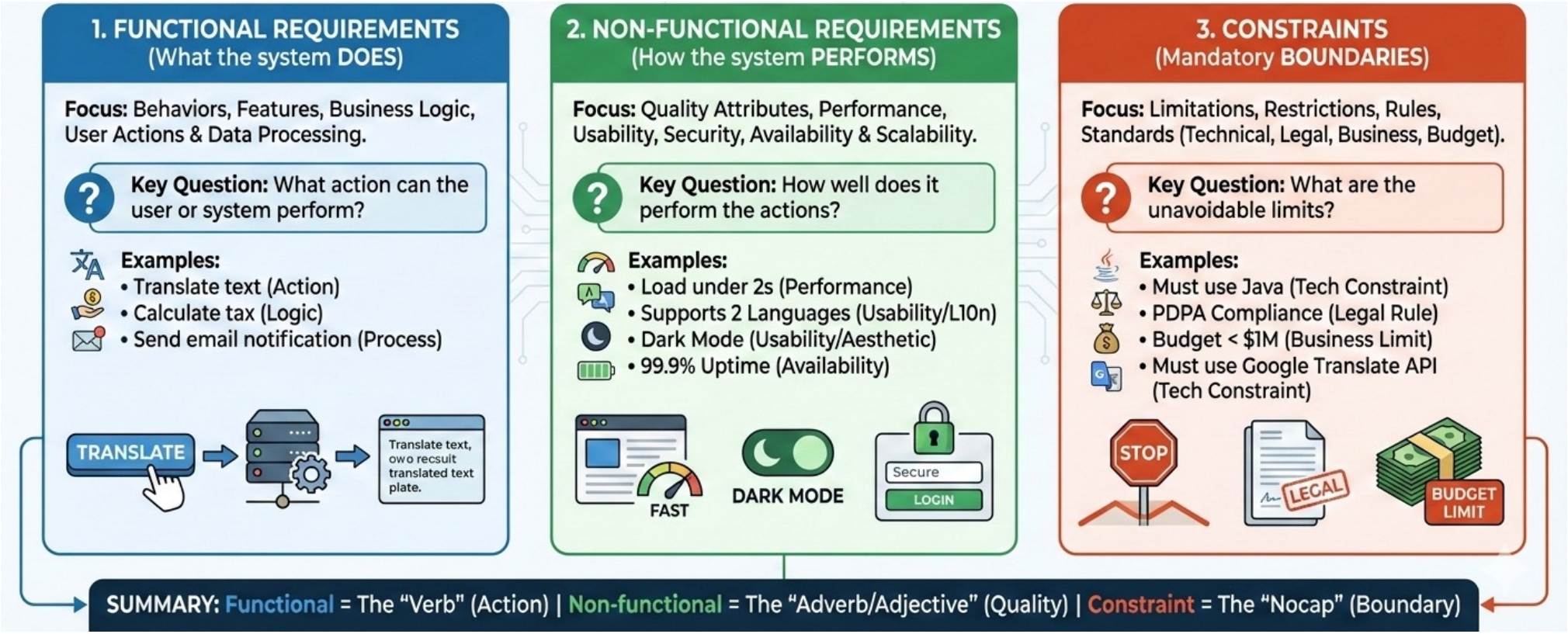
Requirements Verifiability

- Requirements must be written so that they can be objectively verified.
- Imprecise:
  - The system should be easy to use and should be organized in such a way that user errors are minimized.
  - Terms like “easy to use” and “errors shall be minimized” are useless as specifications.
- Verifiable:
  - Users should be able to use the system without having to read a user manual.
  - Users should be able to enter their conflicts for a proposed meeting in under 2 minutes.

Requirements Verifiability

- Verifiability is an important criterion that is typically checked in a formal review of the requirements document.
- Every single requirement must be expressed in such a way that it is **possible to verify that the requirement is fulfilled or not**.
- The classic example of a poorly phrased requirement is that the software should be “easy to use”.
- Since “easy to use” is subjective, it is useless as a formal requirement.
- Instead, the requirements should state explicit, measurable criteria, such as the amount of time it should take a new user to learn it, what the expected throughput of the system should be for a new or an experienced user, or the expected error rate.

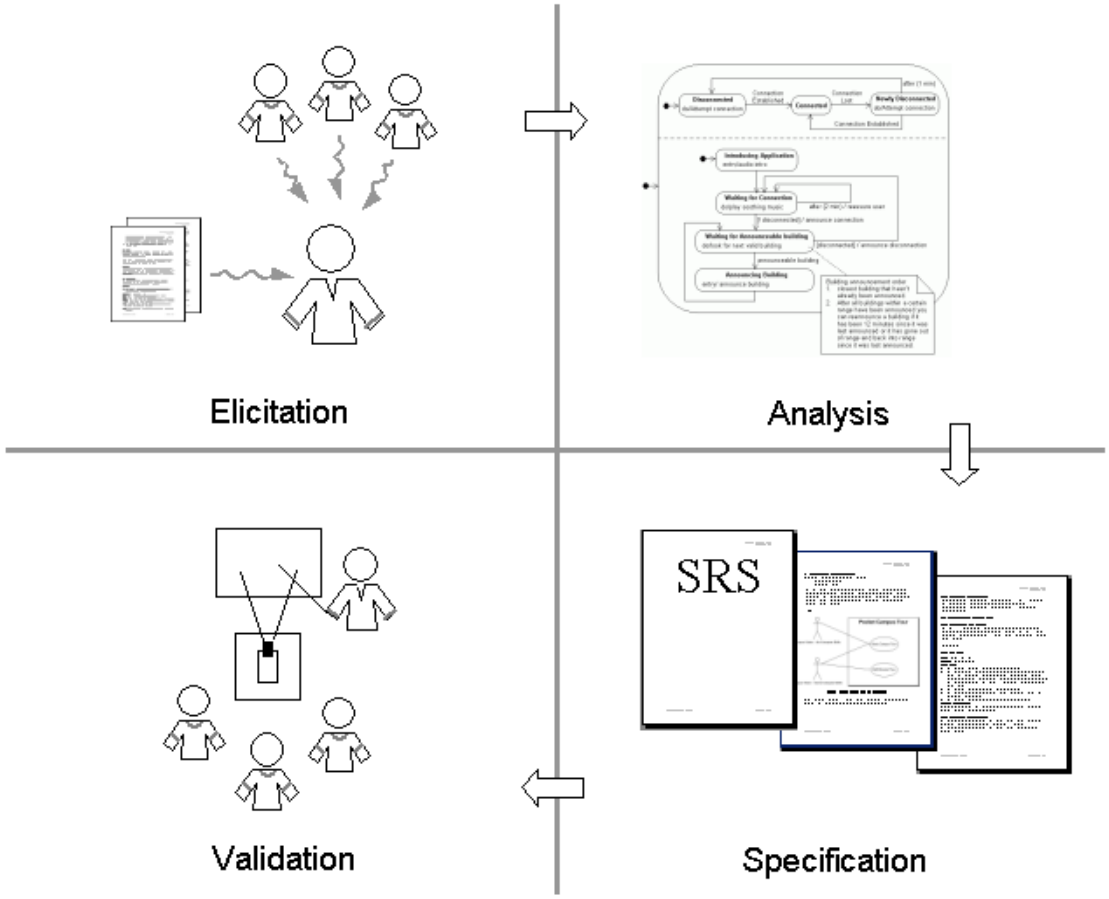
Types of Requirements



Participants in the Requirements Process

- **Customer**—the person who is paying for the software. Even though the customer might not be a user of the system, the customer has significant influence over the requirements.
- **User**—the person that will interact with the application.
- **Product manager/champion**—the person that represents the voice of the customer. This role is especially useful when the customer isn't easily identified or available.
- **Developer**—the person designing and implementing a solution.
- **Analyst (aka Requirements Engineer)**—the person responsible for eliciting the requirements. The liaison between the user/customer and technical staff.

Requirements Development Process



Elicitation

- Study existing written material
- Observe environment
- Interview users and customer
- Help users and customers understand technical possibilities
- Convey relative costs of different implementation options (this will help the customer decide requirements and priorities among requirements)

Interviews

- What should you hear?
  - Wants
  - Need & Requirements
  - Past problems & Complaints
  - Current concerns & criteria
  - Perceptions & Feelings
  - Preferences
  - Ideas & Thoughts
  - Solutions

Analysis

- Elicitation is about gathering requirements, analysis is about **organizing** and **understanding** them better.

- During analysis **priorities** are established, detail is added and missing requirements or unknowns are identified.

- Analysis models may be used to better understand the problem domain and system under development
  - Types of models that might be helpful during requirements analysis include: screen flow diagrams, data flow diagrams, use case models, activity diagrams, and state machine diagrams.

Requirements Checking

Validity	Does the system provide the functions *which best support* the customer's needs?
Consistency	Are there any *requirements conflicts*?
Completeness	Are *all functions* required by the customer included?
Realism	Can the requirements be implemented given *available budget and technology*?

Requirements Reviews

- Regular reviews should be held while the requirements definition is being formulated
- Both client and contractor staff should be involved in reviews
- Reviews may be formal (with completed documents) or informal.
  - Good communications between developers, customers and users can resolve problems at an early stage

Review checks

Verifiability	Is the requirement realistically *testable*?
Comprehensibility	Is the requirement properly *understood*?
Traceability	Is the *origin* of the requirement clearly stated?
Adaptability	Can the requirement be *changed* without a large *impact* on other requirements?

Sample Requirements Review Checklist

- Does the (software) product have a succinct name, and a clearly described purpose?
- Are the characteristics of users and of typical usage mentioned?
  (No user categories missing.)
- Are all external interfaces of the software explicitly mentioned?
  (No interfaces missing.)
- Does each specific requirement have a unique identifier ?
- Is each requirement atomic and simply formulated ?
  (Typically a single sentence. Composite requirements must be split.)
- Are requirements organized into coherent groups ?
  (If necessary, hierarchical; not more than about ten per group.)
- Is each requirement prioritized ?
  (Is the meaning of the priority levels clear?)
- Are all unstable requirements marked as such?
  (TBC=`To Be Confirmed', TBD=`To Be Defined')

Sample Requirements Review Checklist

- Is each requirement verifiable (in a provisional acceptance test)?
  (Measurable: where possible, quantify; capacity, performance, accuracy)
- Are the requirements consistent ? (Non-conflicting.)
- Are the requirements sufficiently precise and unambiguous ?
  (Which interfaces are involved, who has the initiative, who supplies what data, no passive voice.)
- Are the requirements complete? Can everything not explicitly constrained indeed be viewed as developer freedom?
- Is a product that satisfies every requirement indeed acceptable? (No requirements missing.)
- Are the requirements understandable to those who will need to work with them later?
- Are the requirements realizable within budget?
- Do the requirements express actual customer needs (in the language of the problem domain), rather than solutions ?

Traceability

- To protect against changes you should be able to trace back from every system component to the original requirement that caused its presence.
- A software process should help you keep this virtual table up-to-date

	C1	*C2*	…	…	*Cm*
req1		x			
req2	x				
…				x	
…					x
reqn		x	x		

Traceability

- **“Feature creep”** refers to the uncontrolled addition of features to the system.
- To avoid feature creep it is critical to have a clear business vision of what the system should achieve or optimize.
- Traceability helps to keep feature creep under control by tracing new features to the actual requirements they should address.
- In upcoming lectures we will see how agile methods address this issue by guiding both the development team and the customer towards features that bring the most business value to the customer.

Validation

- Use prototypes and other means to verify with users and customer that your understanding of the requirements is complete and correct

Software requirements in three steps

- What are the classes of users?
  - Administrator
  - Supervisor
  - Office Assistant, etc.
- What are the goals associated with each class of user?
  - Enter profile information
  - Edit patient data
  - Query patient identification data, etc.
- What system behavior is needed to accomplish each of the goals for each of the user classes?